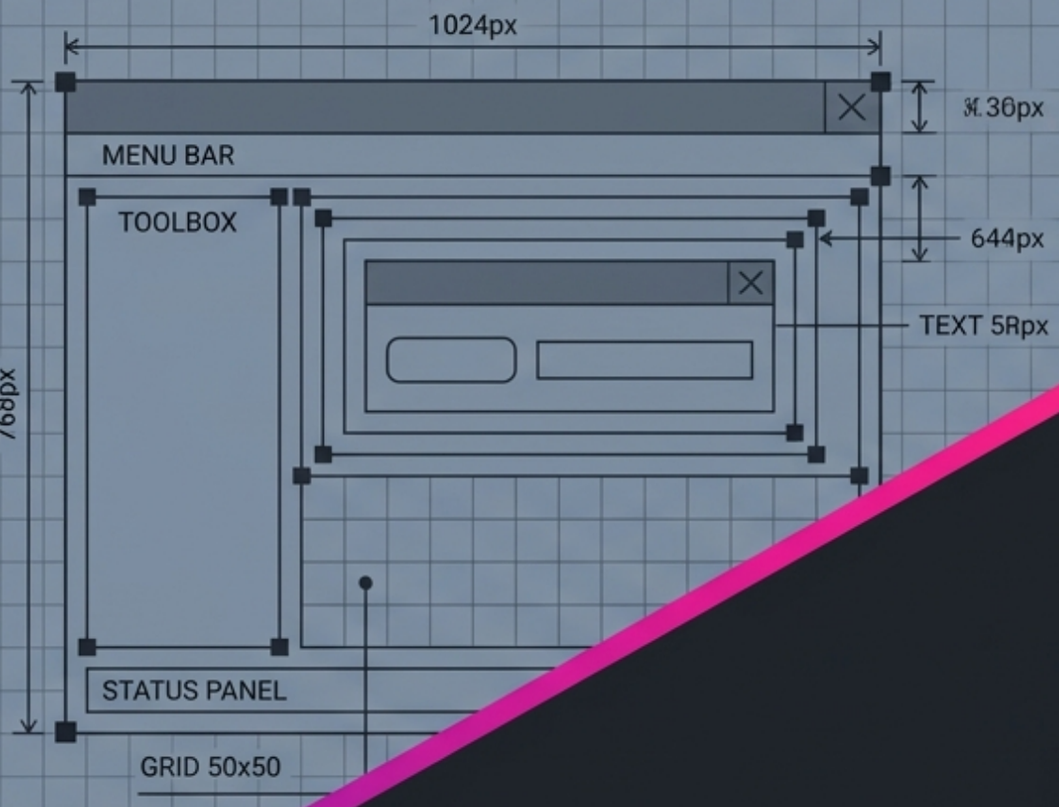
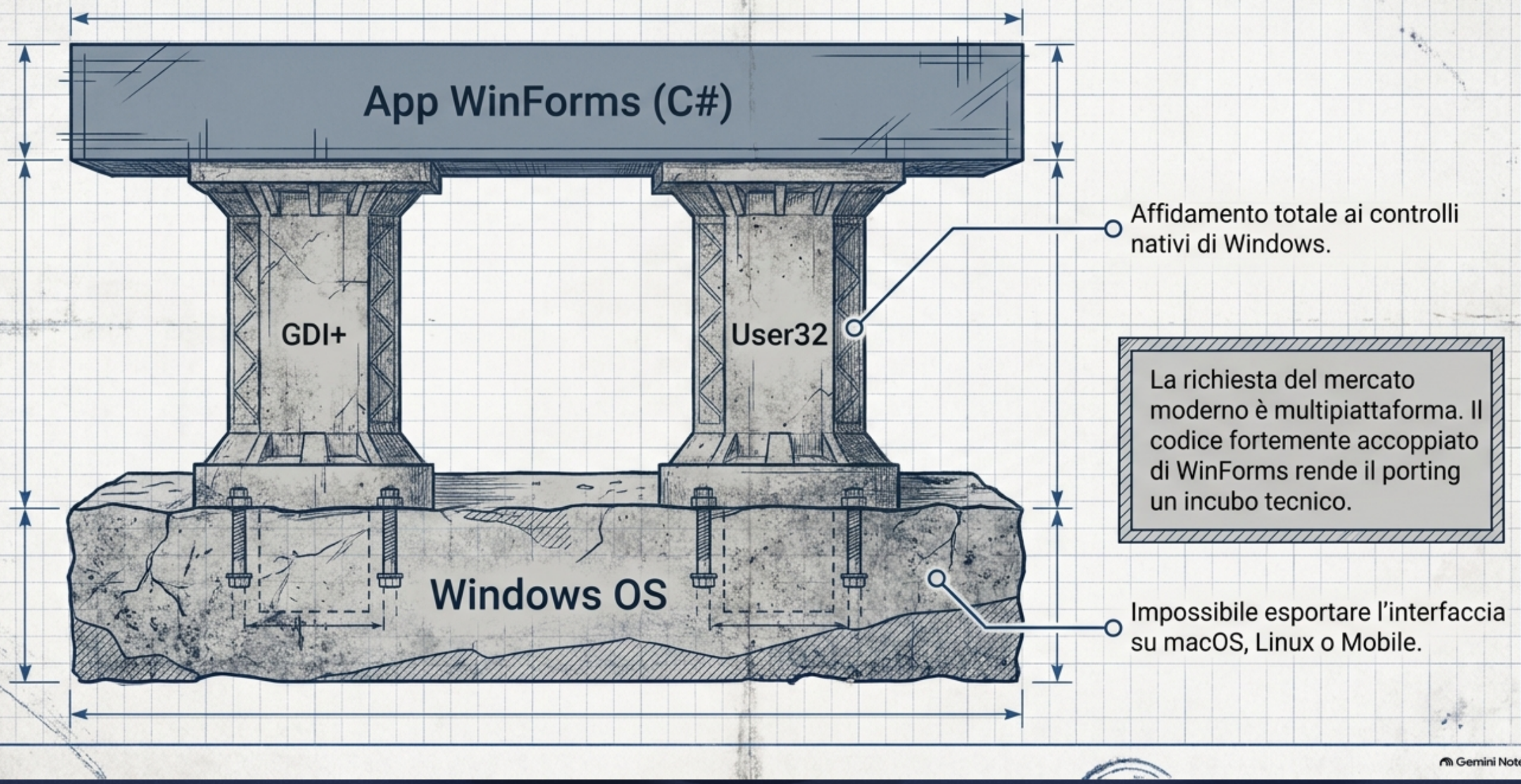
Dal Progetto alla Tela Digitale

Guida visiva ad Avalonia UI per sviluppatori Windows Forms.



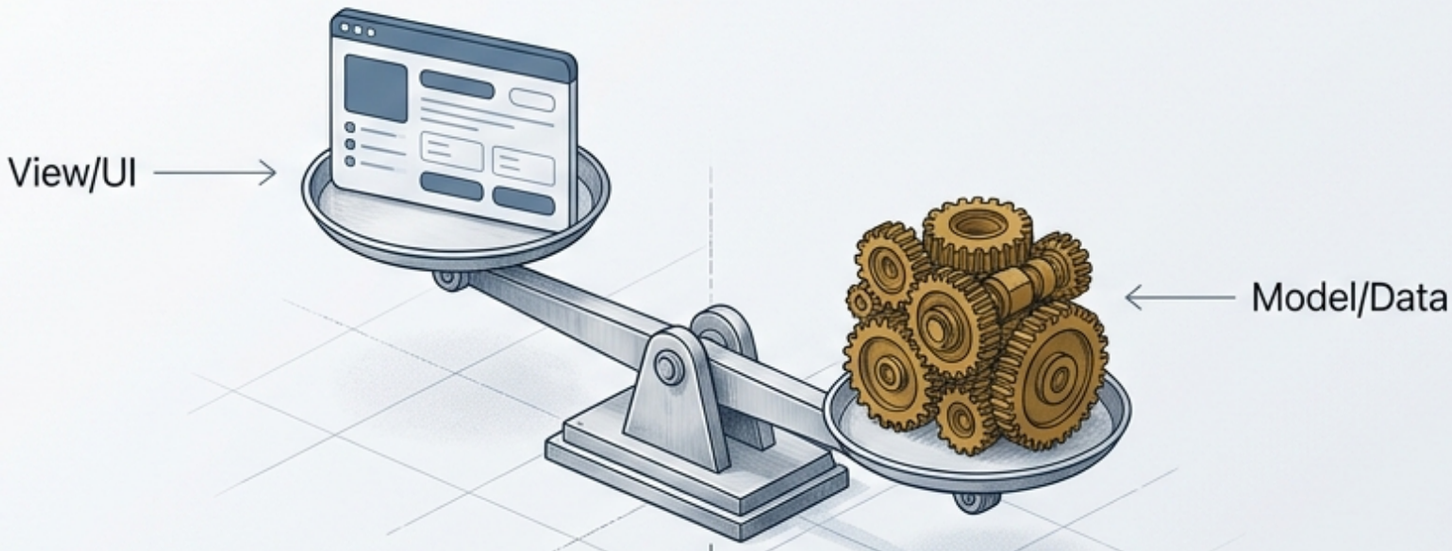
L’architettura WinForms è ancorata a un singolo sistema operativo



Gemini Notebook

MODULAR UNIT A-1

La sfida architetturale è separare rigidamente ciò che l'utente vede da ciò che l'applicazione fa



Disaccoppiamento

Evitare dipendenze dirette tra componenti visivi e logici.

Manutenibilità

Modificare il codice sorgente senza causare regressioni a cascata.

Testabilità

Validare i processi logici senza avviare il motore grafico.

MODULAR UNIT A-1

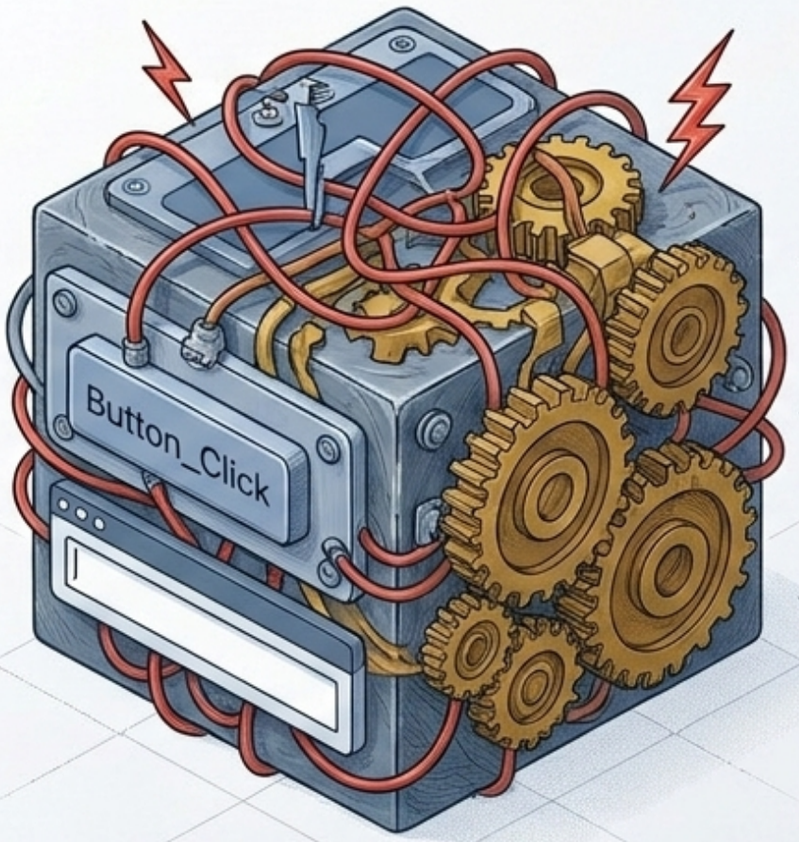
COMPONENT B-2

COMPONENT B-2

SCALE 1:50

Gemini Notebook

Windows Forms vincola la logica direttamente all'interfaccia grafica



WinForms

Paradigma: Event-Driven / Code-Behind (es. Button_Click).

Architettura: La logica applicativa vive nascosta dietro il file di presentazione.

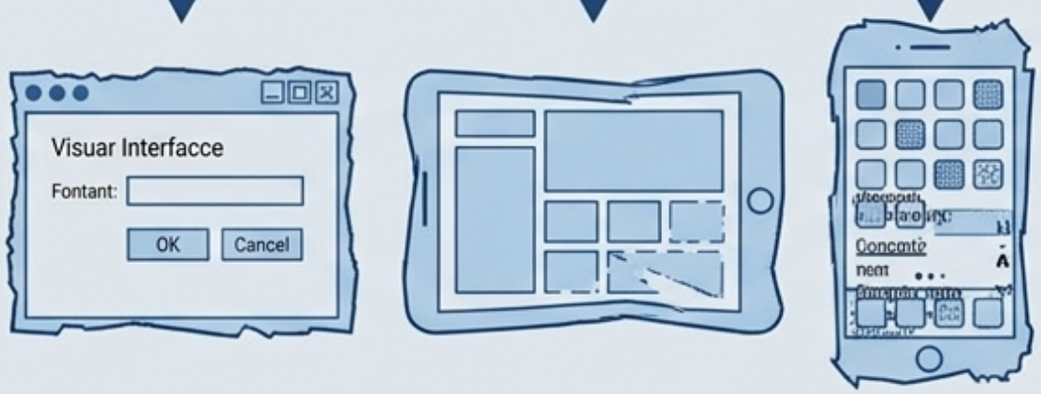
Criticità: Accoppiamento estremo. Genera inevitabilmente "Spaghetti code" nei progetti su larga scala, rendendo impossibile testare la logica senza istanziare l'intera interfaccia grafica.

Il Segreto Architetturale: Rendering Indipendente

Modello Tradizionale

App Code

Wrapper Nativi dell'OS

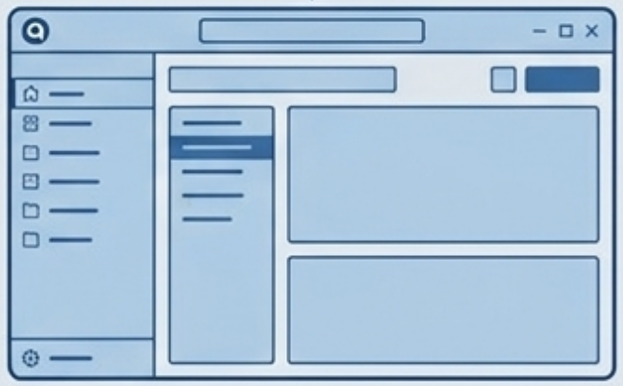


Inconsistenza visiva, limitazioni basate sul sistema operativo e workaround continui.

Modello Avalonia

App Code

Motore Skia / Impeller



Pixel-Perfect. Avalonia non usa i controlli dell'OS. Disegna autonomamente ogni singolo pixel per un'interfaccia identica ovunque.

Nessuna dipendenza dai toolkit nativi. Risultati garantiti e coerenti al 100%.

Gemini Notebook

L’abbandono del Designer in favore del markup AXAML

Il passato: Codice auto-generato, difficile da leggere e prono a conflitti su Git.

```
this.button1.Location = new System.Drawing.Point(10, 10);
this.button1.Name = "button1";
this.button1.Size = new System.Drawing.Size(75, 23);
this.button1.TabIndex = 0;
this.button1.Text = "Cliccami";
this.button1.UsualStatableBackColor = true;
```

Il presente: Markup dichiarativo basato su XML. Definisci cosa vuoi, non come disegnarlo.

```
<StackPanel>
    <Button Content="Cliccami" />
</StackPanel>
```

Attributo 'Content': Definisce cosa viene mostrato all’interno del controllo (testo, un'altra vista, etc.).

Gemini Notebook

Decodificare la struttura di un'interfaccia dichiarativa

Namespace XML: Identifica che questo è un file Avalonia.

Code-Behind Link: Collega questo markup alla tua classe C# (MainWindow.axaml.cs).

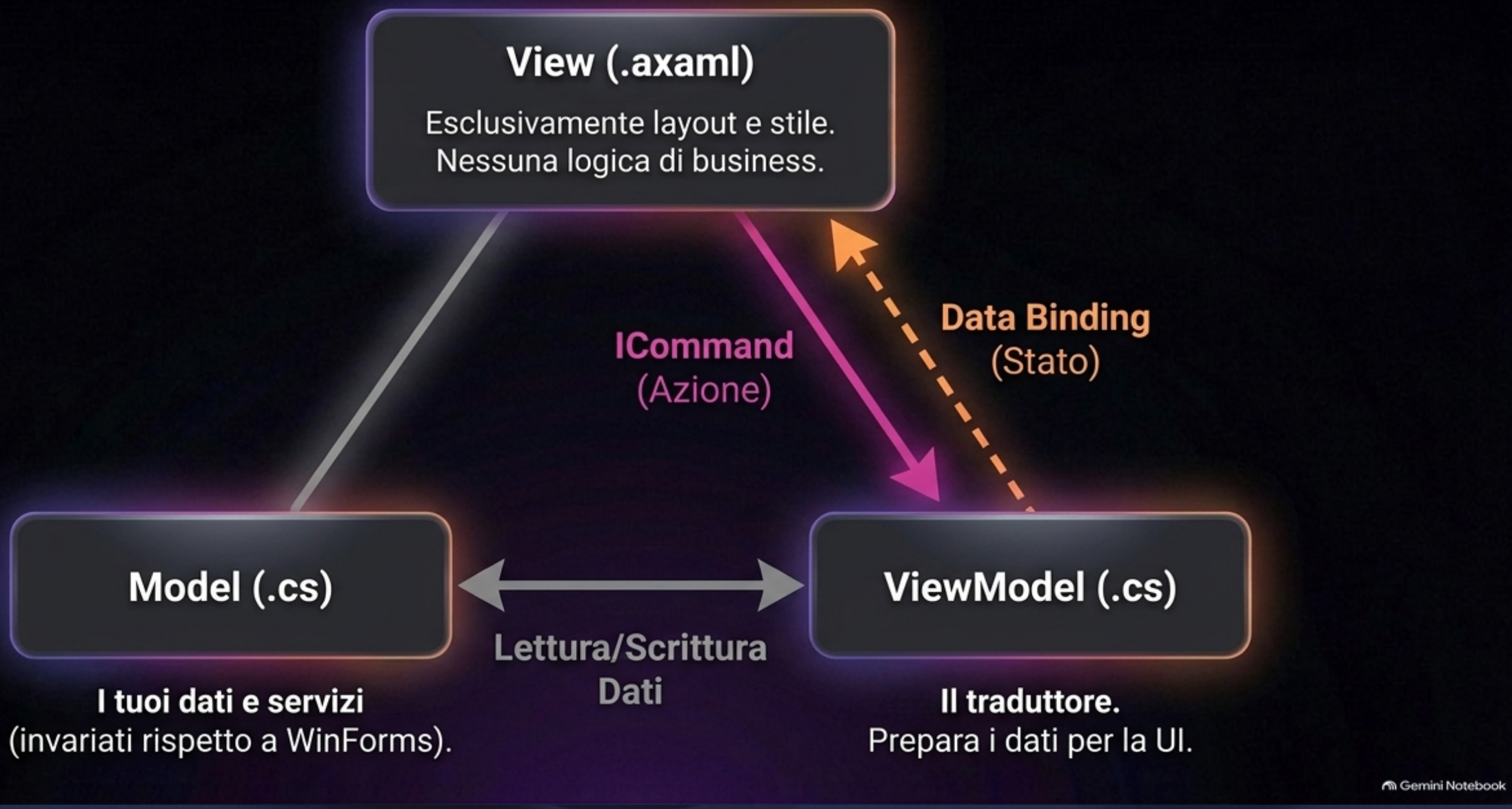
```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="App.MainWindow">
    <Button Background="Blue" Content="{Binding Titolo}" />
</Window>
```

Elemento: Corrisponde esattamente a un oggetto (Classe) in C#.

Attributo: Corrisponde a una Proprietà pubblica della classe.

Gemini Notebook

L’architettura Model-View-ViewModel in azione



Il nostro obiettivo: Cosa costruiremo

Un'interfaccia reattiva per l'inserimento dati e una griglia dinamica, guidata interamente dalla logica del ViewModel.

Gemini Notebook

Passo 1: La Fondazione (Il Model)

Dati puri. Nessuna logica di interfaccia, nessuna dipendenza dal framework UI. La base tipizzata della nostra applicazione.

Person.cs

```
1
2   public class Person
3   {
4       public string FirstName { get; set; }
5       public string LastName { get; set; }
6       public string Email { get; set; }
7   }
```

Gemini Notebook

Passo 2: Il Cervello (ViewModel & ObservableProperty)

La rivoluzione dei Source Generators. L'attributo [ObservableProperty] genera automaticamente il codice per la notifica delle modifiche durante la compilazione.

Person.cs

```
public class OldViewModel : INotifyPropertyChanged {
    private string _inputText;
    public string InputText {
        get => _inputText;
        set {
            if (_inputText != value) {
                _inputText = value;
                OnPropertyChanged(nameof(InputText));
            }
        }
    }
    // ... more boilerplate
}
```

Person.cs

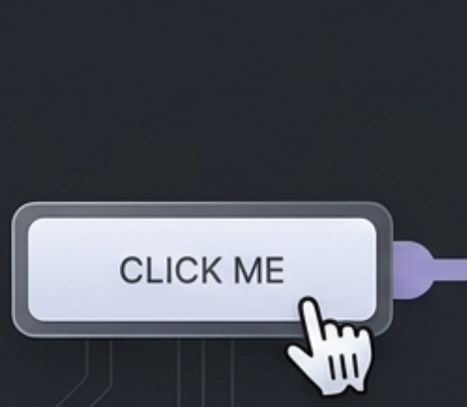
```
public partial class MainWindowViewModel : ObservableObject
{
    [ObservableProperty]
    private string _inputText;

    public ObservableCollection<Person> People { get; } = new();
}
```

Gemini Notebook

Passo 3: L’Azione (Gestire gli Eventi con RelayCommand)

Dimentica gli event handler nel code-behind. [RelayCommand] espone un metodo privato come un comando associabile, pronto per essere invocato dall'interfaccia.




```
[RelayCommand]
private void AddPerson()
{
    if (string.IsNullOrWhiteSpace(InputText)) return;

    People.Add(new Person { FirstName = InputText, LastName = "Utente" });
    InputText = string.Empty; // Resetta l'input
}
```

Gemini Notebook

Passo 4: La Tela (Inizializzare la Window AXAML)

Dichiarare il DataType è fondamentale in Avalonia. Sblocca il Compiled Binding, garantendo controlli a tempo di compilazione e performance di rendering superiori.

Window.ax

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:MyApp.ViewModels"
        x:DataType="vm:MainWindowViewModel"
        x:Class="MyApp.Views.MainWindow">

    <!-- Il contenuto andrà qui -->
</Window>
```

Gemini Notebook

Passo 5: Struttura e Layout Principale

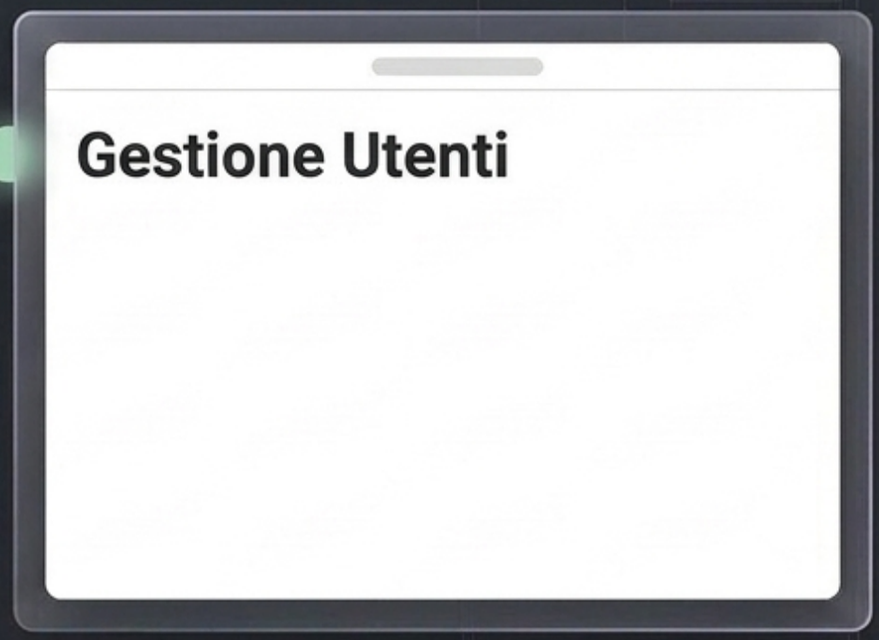
Usa i pannelli di layout come StackPanel per organizzare lo spazio. Il TextBlock renderizza il testo statico o collegato in sola lettura.

Window.ax

```
<StackPanel Margin="20" Spacing="15">
    <TextBlock Text="Gestione Utenti" FontSize="24"
          FontWeight="Bold" />

    <!-- Area Input -->

    <!-- Area Griglia -->
</StackPanel>
```



Gemini Notebook

Passo 6: L’Input dell’Utente (Two-Way Binding)

Interazione perfetta. Il TextBox aggiorna il ViewModel istantaneamente, mentre il Button invoca il nostro RelayCommand generato precedentemente.

Window.ax

```
<Grid ColumnDefinitions="*, Auto">
    <TextBox Grid.Column="0"
             Text="{Binding InputText, Mode=TwoWay}"
             Watermark="Inserisci il nome..." />

    <Button Grid.Column="1" Margin="10,0,0,0"
             Command="{Binding AddPersonCommand}"
             Content="Aggiungi" />
</Grid>
```

Gemini Notebook

Passo 7: Visualizzare i Dati (Il DataGrid)

Collega la proprietà ItemsSource alla tua ObservableCollection. Avalonia gestisce dinamicamente le righe e le colonne reagendo in tempo reale ad ogni aggiunta.

Window.ax

```
<DataGrid ItemsSource="{Binding People}"
          AutoGenerateColumns="True"
          IsReadOnly="True"
          GridLinesVisibility="All"
          BorderThickness="1"
          BorderBrush="Gray">
</DataGrid>
```

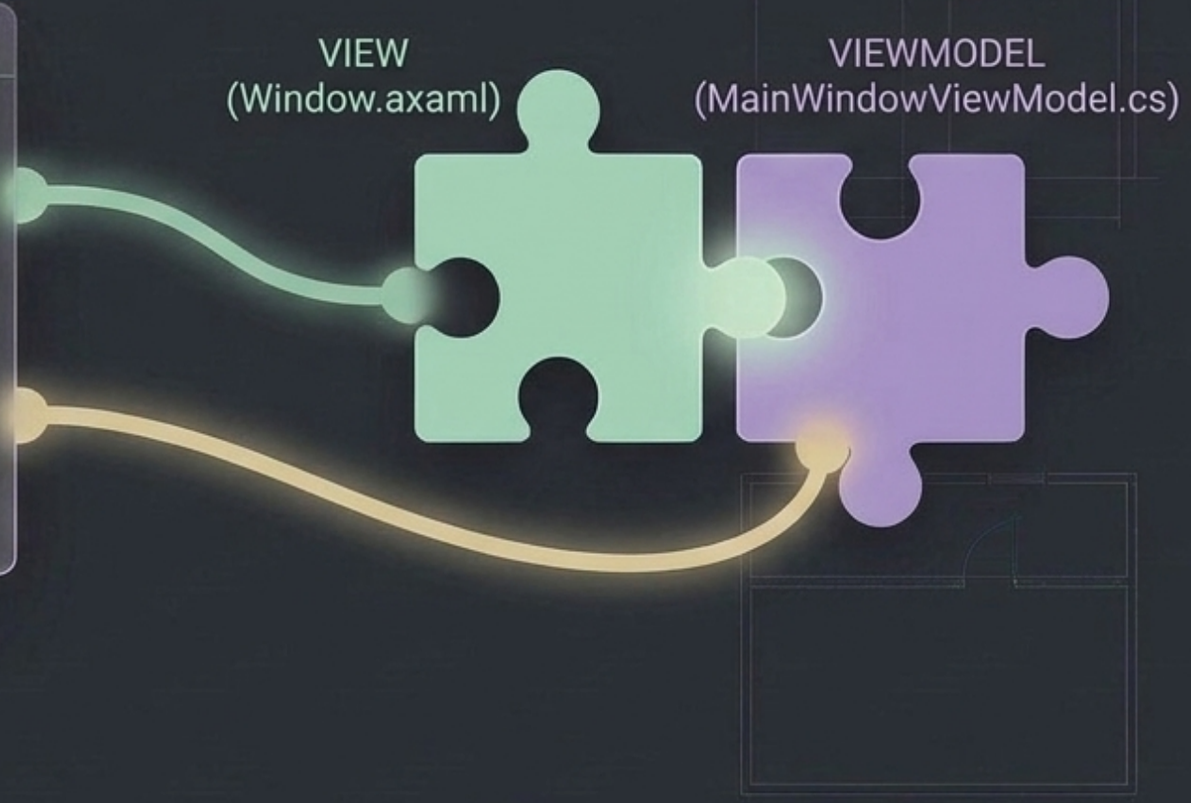
Gemini Notebook

Passo 8: L’Integrazione (Il Code-Behind)

L’ultimo tassello per dare vita al binding. Istanzia il ViewModel e assegnalo al DataContext della Window direttamente nel costruttore.

MainWindow.axaml.cs

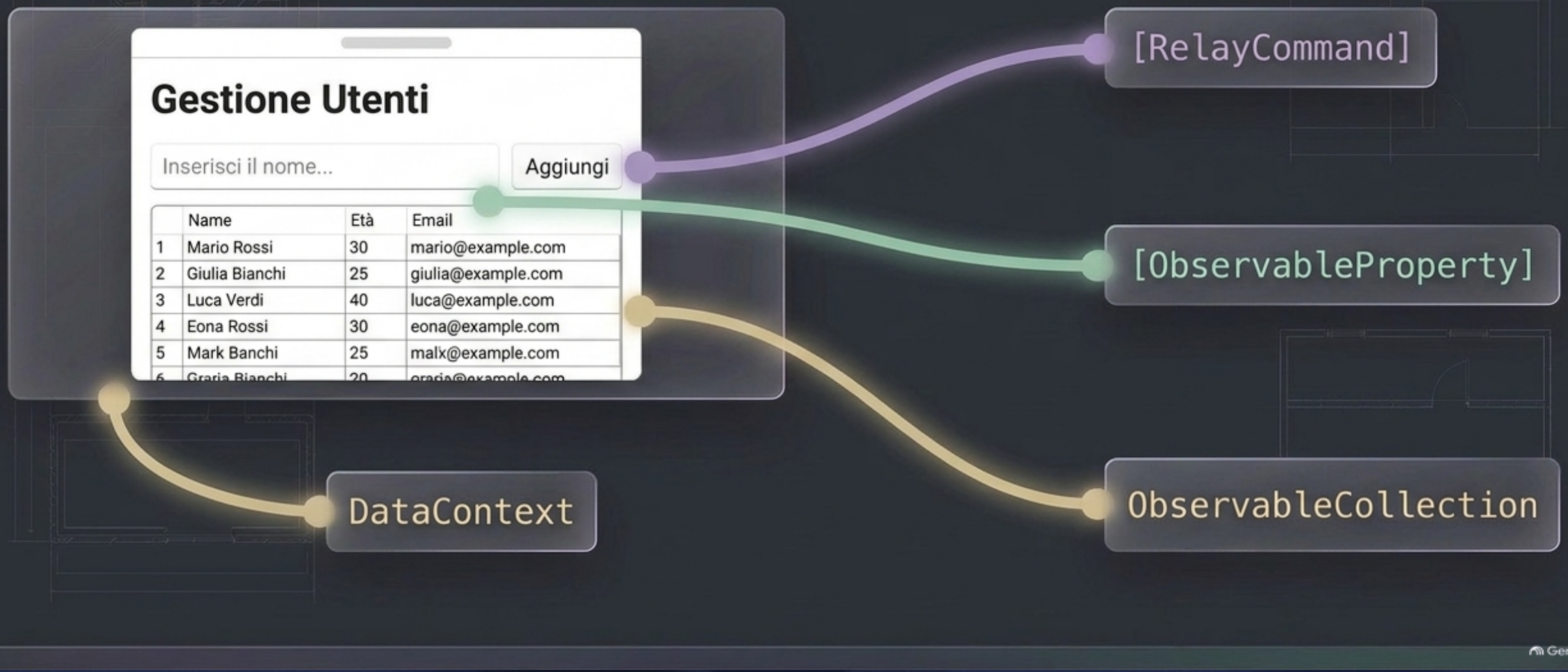
```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new MainWindowViewModel();
    }
}
```



Gemini Notebook

Sintesi: L’Ecosistema in Azione

Un flusso di lavoro elegante e tipizzato. Il design (AXAML) è isolato dalla logica (C#), ma tutto è perfettamente sincronizzato e reattivo grazie al CommunityToolkit.Mvvm.



Oltre Windows: Il Vantaggio Strategico

Scritto una volta. Renderizzato da Skia ovunque. Ottieni un'interfaccia pixel-perfect e nativa su Windows, macOS e Linux, senza alcuna modifica alla tua codebase.

Sei pronto per il futuro dello sviluppo UI .NET.

Installa i template, esplora la documentazione ufficiale e inizia a costruire interfacce multipiattaforma senza compromessi.

Documentazione Avalonia UI

docs.avaloniaui.net

CommunityToolkit.Mvvm Repo

github.com/CommunityToolkit/dotnet

Gemini Notebook